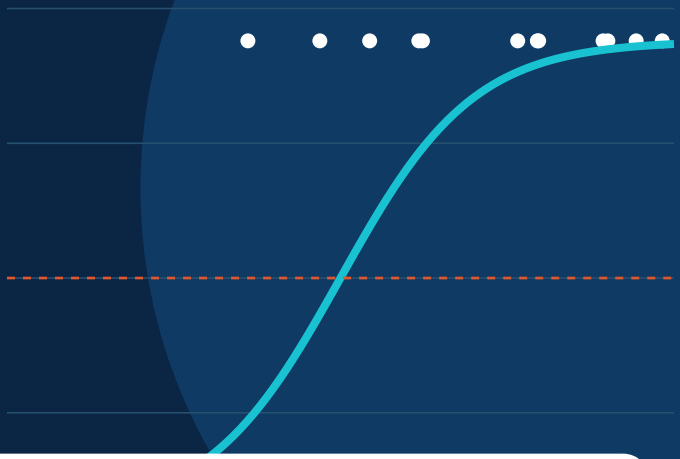
MACHINE LEARNING · STUDY GUIDE

Logistic Regression Analysis

From first principles to regularized, multinomial, ordinal, Firth, Bayesian and 20+ related models, explained with executed Python code and line-by-line walkthroughs.



PREPARED BY

Chandan Chaudhari

GitHub: github.com/chandanc5525

Contents

1. Introduction 4

1.1 Types of logistic regression 4

1.2 Why it matters 4

1.3 Python toolkit used in these notes 5

2. Intuition 5

2.1 The idea in four steps 6

2.2 Three mental pictures 6

3. Mathematics 7

3.1 Sigmoid, odds and logit 7

3.2 Likelihood and the loss function 7

3.3 Gradient 8

3.4 Optimisation 8

3.5 Decision boundary 8

3.6 Multinomial (softmax) extension 8

3.7 Regularization 9

3.8 Assumptions 9

4. Evaluation Metrics 10

4.1 Metrics from the confusion matrix 10

4.2 Probability-based metrics 11

4.3 Choosing the threshold 11

4.4 Worked example 12

4.5 Computing metrics in Python 12

4.6 Checking the assumptions 14

5. Use Cases 15

5.1 What logistic regression does best 15

5.2 When to use and when to avoid 15

5.3 Case study: cross-validation and odds ratios 16

6. Python Code Structure for Classification Projects 17

6.1 The estimator API for classifiers 17

6.2 Key LogisticRegression parameters . . . 18
6.3 Recommended project workflow . . . 18
6.4 A reusable evaluation helper . . . 18

7. Regularized Logistic Regression: L2, L1 and ElasticNet . . . 19

8. More Models in the Logistic Family . . . 21

8.1 Class-weighted logistic regression (imbalanced data) . . . 21
8.2 Multinomial logistic regression (softmax) . . . 22
8.3 One-vs-Rest and multilabel classification . . . 23
8.4 Ordinal logistic regression (proportional odds) . . . 23
8.5 Polynomial and spline logistic regression . . . 24
8.6 Kernel logistic regression (Nystroem approximation) . . . 25
8.7 Probit and complementary log-log models . . . 25
8.8 Firth penalized logistic regression . . . 26
8.9 Conditional logistic regression . . . 27
8.10 Bayesian logistic regression (Laplace approximation) . . . 28
8.11 SGDClassifier and online learning . . . 28
8.12 Automatic tuning: LogisticRegressionCV and GridSearchCV . . . 29
8.13 Probability calibration . . . 29
8.14 Feature selection: SelectFromModel and RFECV . . . 30
8.15 Related linear classifiers . . . 31

9. Model Comparison and How to Choose . . . 32

9.1 Decision guide . . . 34
9.2 Master comparison of the penalized family . . . 34

10. Cheat Sheet and Summary . . . 35

Learning roadmap . . . 35

Document map

Sections 1–6 cover introduction, intuition, mathematics, evaluation metrics, use cases and the Python code structure. Sections 7–8 explain L2, L1 and ElasticNet logistic regression plus more than twenty further models, each with runnable Python code. Section 9 compares them on one dataset.

1. Introduction

Logistic Regression is a **supervised learning** algorithm for **classification**. Despite the word "regression" in its name, it predicts the **probability** that an observation belongs to a class, most often a binary outcome such as yes/no, spam/not spam or default/no default. A final class label is obtained by comparing that probability with a threshold (0.5 by default).

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Term	Meaning
Target y	Categorical outcome coded 0 (negative class) or 1 (positive class)
Features $x_1 \ldots x_p$	Numeric or encoded inputs used to explain the outcome
z (log-odds / logit)	The linear score $\beta_0 + \beta^T x$; can be any real number
Sigmoid σ(z)	S-shaped function that squeezes any score into a probability between 0 and 1
Odds and odds ratio	odds = p/(1−p); $\exp(\beta_j)$ is the factor by which the odds change for a one-unit increase in x_j
Threshold	Probability cut-off that converts p into a class label; tune it to the cost of errors

1.1 Types of logistic regression

Type	Outcome	Example
Binary	Two classes	Fraud / not fraud
Multinomial (softmax)	3+ unordered classes	Cat / dog / bird
Ordinal (proportional odds)	3+ ordered classes	Low < medium < high risk
Multilabel (one-vs-rest)	Several independent yes/no labels	Tags on a news article
Regularized	Any of the above plus an L1/L2 penalty	Many correlated features
Specialised	Rare events, matched data, small samples	Firth, conditional, Bayesian logistic

1.2 Why it matters

- **Probabilities, not just labels:** scores can be ranked, thresholded, priced and combined with business costs.
- **Interpretable:** each coefficient converts into an odds ratio, which is why banks, hospitals and regulators rely on it.
- **Fast and data-efficient:** trains in milliseconds, works with modest data and is a very strong baseline.
- **Foundation of modern ML:** a single neuron with a sigmoid is logistic regression; softmax regression is the output layer of neural-network classifiers.

1.3 Python toolkit used in these notes

Library	Role
NumPy	Arrays and linear algebra; used to build logistic regression from scratch
pandas	Tabular data and readable coefficient tables
scikit-learn	LogisticRegression, SGDClassifier, pipelines, cross-validation, metrics, calibration
statsmodels	Statistical inference (p-values, odds ratios), probit/cloglog GLMs, ordinal and conditional logit
SciPy	Optimization and statistical distributions
Matplotlib	ROC curves, decision boundaries, coefficient paths

How to read the code in this document

Every Python block is **executed for real** while this PDF is generated, so each **OUTPUT** panel is genuine. Blocks share one session in order, so later blocks reuse earlier variables. Each block is followed by a **Code walkthrough** table explaining the lines in plain language. Install with `pip install numpy pandas scikit-learn statsmodels scipy matplotlib`. The regularization code uses the `l1_ratio` argument of scikit-learn 1.8 or newer.

2. Intuition

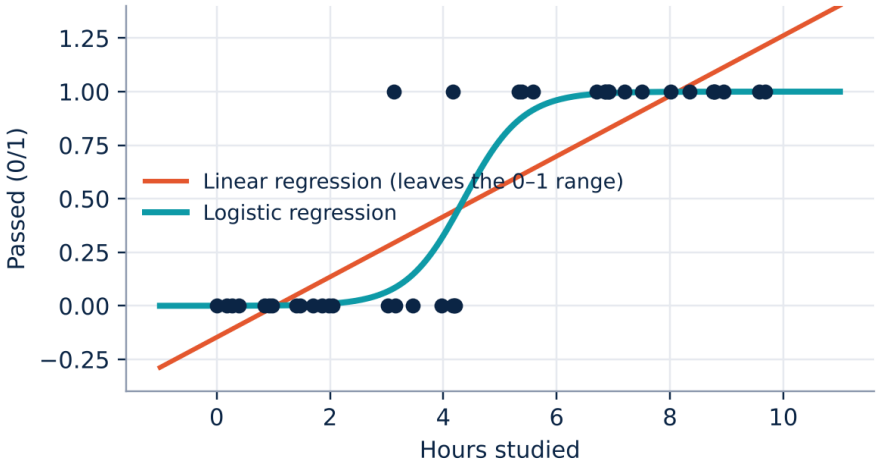


Figure 1. A straight line escapes the 0–1 range; the logistic curve stays a valid probability.

Suppose we predict whether a student passes (1) or fails (0) from hours studied. A straight line fitted to 0/1 data predicts values below 0 and above 1, which are meaningless as probabilities, and it is dragged around by extreme points. We need a curve that rises smoothly from 0 to 1: the **sigmoid**.

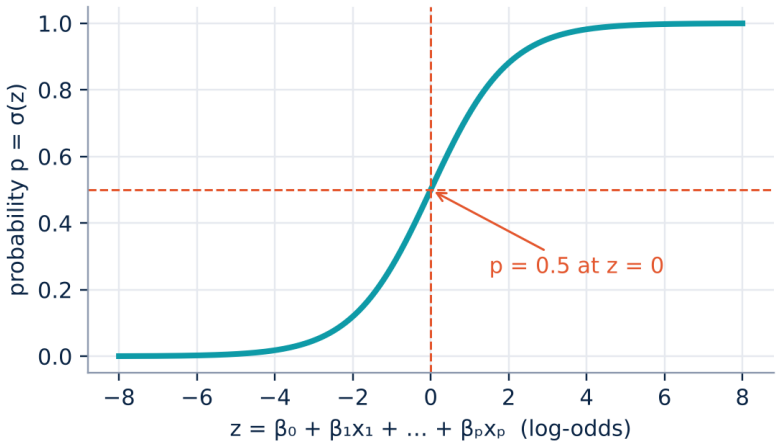


Figure 2. The sigmoid turns any score z into a probability; z = 0 gives exactly 0.5.

2.1 The idea in four steps

- **Score:** combine the features linearly, $z = \beta_0 + \beta_1 x_1 + \ldots$ (any real number).
- **Squash:** pass z through the sigmoid to obtain a probability between 0 and 1.
- **Decide:** predict class 1 when p ≥ threshold (default 0.5), otherwise class 0.
- **Learn:** choose the β values that make the observed labels as likely as possible (maximum likelihood).

2.2 Three mental pictures

- **Confidence slider:** a large positive z means “almost certainly 1”, a large negative z means “almost certainly 0”, and z = 0 means “toss-up”.
- **Boundary and distance:** the points where p = 0.5 form a straight line (or plane). The farther a point lies from this boundary, the more confident the prediction.
- **Odds multiplier:** every +1 in a feature multiplies the odds of class 1 by e^{β}. With β = 0.7 the odds roughly double.

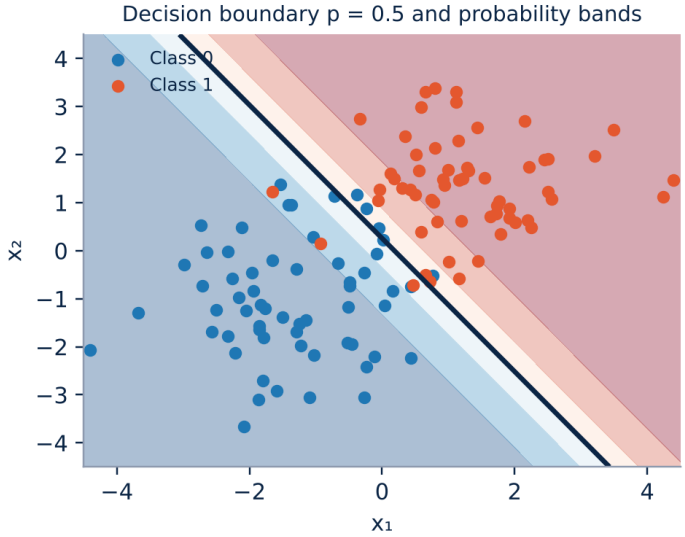


Figure 3. Decision boundary (p = 0.5) of a fitted logistic regression; colours show the probability.

Key insight

Learning minimizes **log loss**: the penalty is enormous when the model is confident *and wrong* and tiny when it is confident and right. This pushes the model toward honest, well-calibrated probabilities.

PYTHON The sigmoid in Python

```python
import numpy as np
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
z = np.array([-6, -2, 0, 2, 6])
print('score  z :', z)
print('probability:', sigmoid(z).round(3))
```

OUTPUT

```
score  z : [-6 -2  0  2  6]
probability: [0.002 0.119 0.5   0.881 0.998]
```

Code walkthrough

Lines	What the code does and why
1–3	Import NumPy and define the sigmoid $1 / (1 + e^{-z})$ so that it works on whole arrays at once.
4–6	Evaluate the function at five scores. z = 0 gives exactly 0.5, while large |z| saturates close to 0 or 1.

3. Mathematics

3.1 Sigmoid, odds and logit

$$\sigma(z) = \frac{1}{1+e^{-z}} \qquad \text{odds} = \frac{p}{1-p} \qquad \text{logit}(p) = \ln\frac{p}{1-p} = \beta_0 + \boldsymbol{\beta}^T\mathbf{x}$$

Logistic regression is therefore **linear in the log-odds**. Each coefficient has a direct meaning: holding other features fixed, a one-unit increase in x_j adds β_j to the log-odds, or multiplies the odds by the **odds ratio**:

$$\text{OR}_j = e^{\beta_j} \qquad \left(\beta_j = 0.7 \;\Rightarrow\; e^{0.7} \approx 2.01 : \text{ odds double}\right)$$

3.2 Likelihood and the loss function

Each label is Bernoulli-distributed with success probability p_i. Assuming independent observations, the likelihood is the product of the individual probabilities. Taking the logarithm and negating gives the **binary cross-entropy (log loss)**:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{\,y_i}(1-p_i)^{\,1-y_i}$$

$$J(\boldsymbol{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \ln p_i + (1-y_i)\ln(1-p_i)], \qquad p_i = \sigma(\mathbf{x}_i^T\boldsymbol{\beta})$$

This loss is **convex**, so it has a single global minimum. (Using squared error with a sigmoid would create a non-convex surface with flat regions.)

3.3 Gradient

Using the identity σ(z) = σ(z)(1 − σ(z)), the derivative simplifies beautifully:

$$\nabla J(\boldsymbol{\beta}) = \tfrac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y}), \qquad \mathbf{p} = \sigma(\mathbf{X}\boldsymbol{\beta})$$

This has exactly the same shape as the linear-regression gradient, except that the prediction is σ(Xβ) instead of Xβ.

3.4 Optimisation

Unlike linear regression there is **no closed-form solution**, because setting the gradient to zero gives a non-linear equation. We solve it iteratively:

$$\text{Gradient descent:} \quad \boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \alpha\, \nabla J$$

$$\text{Newton / IRLS:} \quad \boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^T(\mathbf{y} - \mathbf{p}), \quad W_{ii} = p_i(1 - p_i)$$

Method	Idea	Used by
Gradient descent / SGD	Small steps along the negative gradient	SGDClassifier, saga, sag
Newton–Raphson / IRLS	Uses curvature (Hessian); converges in very few iterations	statsmodels, newton-cg, newton-cholesky
L-BFGS	Quasi-Newton: approximates curvature cheaply	scikit-learn default (lbfgs)
Coordinate descent	Updates one coefficient at a time; handles L1	liblinear

3.5 Decision boundary

$$\hat{y} = 1 \iff p \geq 0.5 \iff z = \beta_0 + \boldsymbol{\beta}^T\mathbf{x} \geq 0$$

The boundary is the set of points where z = 0: a straight line in two dimensions and a hyperplane in higher dimensions. Changing the threshold shifts the boundary parallel to itself.

3.6 Multinomial (softmax) extension

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad J = -\tfrac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \ln p_{ik}$$

3.7 Regularization

$$J_{L2} = J + \frac{\lambda}{2}\sum_j \beta_j^2 \qquad\qquad J_{L1} = J + \lambda\sum_j |\beta_j|$$

In scikit-learn the strength is **C = 1/λ**, so a **small C means strong regularization**. The intercept is not penalized.

3.8 Assumptions

Assumption	Meaning	How to check
Binary / properly coded outcome	Classes are mutually exclusive and correctly labelled	Inspect class counts
Independent observations	No repeated or clustered rows (otherwise use mixed or conditional models)	Study design, grouping variables
Linearity of the logit	Each numeric feature is linearly related to the log-odds	Binned log-odds plots, add splines or squares
No severe multicollinearity	Features are not near-duplicates	VIF > 5–10, correlation matrix
Sufficient events	About 10+ events of the rarer class per feature	Count positives vs features
No extreme outliers	Influential points can distort coefficients	Residuals, Cook's distance

Not required (unlike linear regression): normally distributed features, normal residuals or constant variance.

PYTHON Logistic regression from scratch: gradient descent vs Newton

```python
rng = np.random.RandomState(0)
X0 = rng.normal(size=(200, 2))
y0 = (X0[:, 0] + 2*X0[:, 1] + rng.normal(0, .5, 200) > 0).astype(int)
Xb0 = np.c_[np.ones(len(X0)), X0]
beta = np.zeros(3); alpha = 0.5
for i in range(2001):
    p = sigmoid(Xb0 @ beta)
    beta -= alpha * Xb0.T @ (p - y0) / len(y0)
    if i % 1000 == 0:
        loss = -np.mean(y0*np.log(p) + (1-y0)*np.log(1-p))
        print(f'iter {i:4d}  log-loss = {loss:.4f}')
beta2 = np.zeros(3)
for _ in range(8):
    p = sigmoid(Xb0 @ beta2)
    H = Xb0.T @ (Xb0 * (p * (1 - p))[:, None])
    beta2 += np.linalg.solve(H, Xb0.T @ (y0 - p))
print('gradient descent:', beta.round(2))
print('Newton / IRLS   :', beta2.round(2))
```

OUTPUT

```
iter    0  log-loss = 0.6931
iter 1000  log-loss = 0.1623
iter 2000  log-loss = 0.1616
gradient descent: [-0.23  3.39  6.8 ]
Newton / IRLS   : [-0.24  3.47  6.96]
```

Code walkthrough

Lines	What the code does and why
1–4	Create 200 synthetic points with two features and a label from a noisy linear rule. A column of ones is prepended so the intercept is learned like any weight.
5–11	Gradient descent: compute probabilities, apply the gradient $X^T(p - y)/n$ and step downhill. The log loss printed every 1000 iterations keeps falling.
12–16	Newton / IRLS: build the weighted matrix X^TWX (the Hessian) and solve for the step. Eight iterations are plenty.
17–18	Both methods reach similar coefficients; Newton gives the exact optimum while gradient descent is still creeping toward it.

4. Evaluation Metrics

Accuracy alone can be dangerously misleading: a model that always predicts “not fraud” is 99% accurate when only 1% of transactions are fraudulent. Evaluation starts with the **confusion matrix**, which counts every combination of actual and predicted class.

	Predicted 1 (positive)	Predicted 0 (negative)
Actual 1	TP: true positive (correct alarm)	FN: false negative (missed case)
Actual 0	FP: false positive (false alarm)	TN: true negative (correct rejection)

4.1 Metrics from the confusion matrix

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \qquad \text{Precision} = \frac{TP}{TP+FP} \qquad \text{Recall} = \frac{TP}{TP+FN}$$

$$\text{Specificity} = \frac{TN}{TN+FP} \qquad F_1 = \frac{2PR}{P+R} \qquad F_\beta = (1+\beta^2)\frac{PR}{\beta^2 P+R}$$

$$\text{MCC} = \frac{TP\cdot TN - FP\cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Metric	Question it answers	Use it when
Precision	Of everything flagged positive, how much is truly positive?	False alarms are costly (spam filter, fraud review queue)
Recall / sensitivity	Of all real positives, how many did we catch?	Misses are costly (disease screening, fraud losses)
Specificity	Of all real negatives, how many were correctly cleared?	Pair with recall to describe both error types
F1 / Fβ	Single balance of precision and recall (β > 1 favours recall)	Imbalanced classes, one headline number
MCC	Correlation between predicted and actual labels, in [−1, 1]	Imbalanced data; uses all four cells
Accuracy	Overall fraction correct	Balanced classes only

4.2 Probability-based metrics

$$\text{LogLoss} = -\frac{1}{n}\sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)] \qquad \text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2$$

- **ROC curve / AUC:** plots true-positive rate against false-positive rate over all thresholds. AUC is the probability that a random positive is ranked above a random negative: 0.5 = coin flip, 1.0 = perfect ranking.
- **PR curve / average precision:** precision against recall; more informative than ROC when positives are rare.
- **Log loss and Brier score:** judge the quality of the probabilities themselves (lower is better). They punish over-confident mistakes.
- **Calibration:** do predictions of 0.8 come true about 80% of the time? Logistic regression is usually well calibrated; Naive Bayes and SVMs are not.

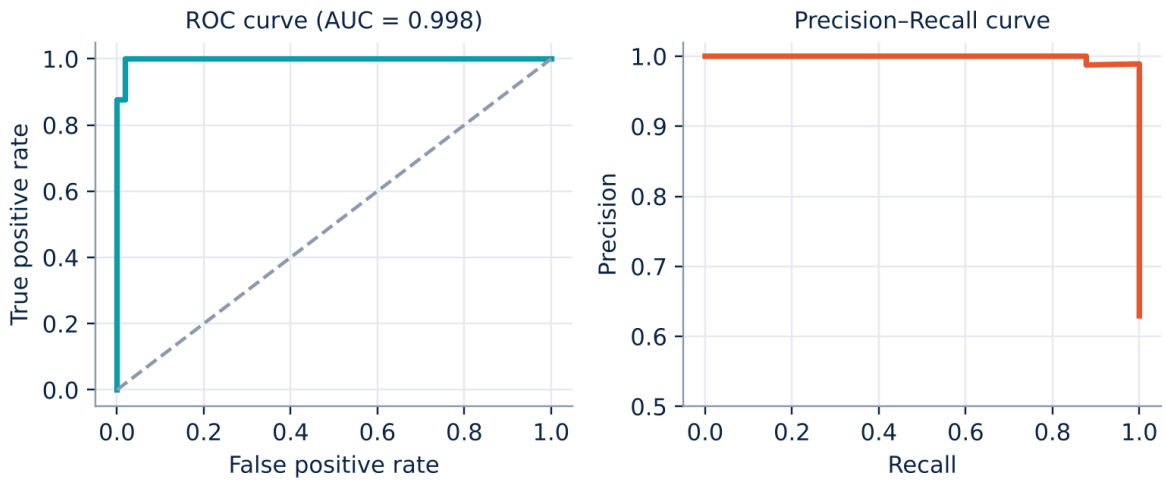


Figure 4. ROC and precision–recall curves of the logistic model on the held-out breast-cancer data.

4.3 Choosing the threshold

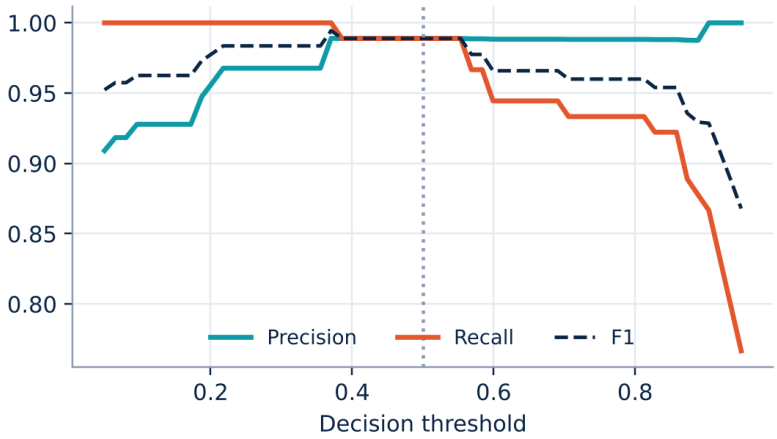


Figure 5. Raising the threshold increases precision and lowers recall.

The 0.5 threshold is only a default. For disease screening lower it to catch every case; for a spam filter raise it so real email is never lost. Pick the threshold from the business cost of false positives versus false negatives, using a validation set.

4.4 Worked example

TP = 40, FP = 10, FN = 20, TN = 130 (n = 200):

Metric	Calculation	Value
Accuracy	(40 + 130) / 200	0.850
Precision	40 / (40 + 10)	0.800
Recall	40 / (40 + 20)	0.667
Specificity	130 / (130 + 10)	0.929
F1	2 × 0.800 × 0.667 / (0.800 + 0.667)	0.727
MCC	(40×130 − 10×20) / √(50×60×140×150)	0.630

Accuracy looks healthy at 85%, yet one third of the real positives are missed (recall 0.667): exactly why several metrics are needed.

4.5 Computing metrics in Python

PYTHON Load data, split, fit and predict

```python
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print('shape:', X.shape, ' classes:', y.value_counts().to_dict())
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]
pred = (proba >= 0.5).astype(int)
```

OUTPUT

```
shape: (569, 30)  classes: {1: 357, 0: 212}
```

Code walkthrough

Lines	What the code does and why
1–6	Import the data loader, splitter, scaler and the `LogisticRegression` class.
7–9	Load 569 tumours with 30 measurements. Target 1 = benign, 0 = malignant. Always check the class balance first.
10–11	Hold out 25% of the rows. `stratify=y` keeps the class ratio identical in both parts, which is essential for classification.
12–16	Scale features inside a pipeline (no leakage) and fit. `predict_proba` returns [P(0), P(1)] per row; column 1 is P(benign). Applying the 0.5 threshold gives hard labels.

PYTHON Every metric in one place

```python
from sklearn.metrics import (confusion_matrix, accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score,
    average_precision_score, log_loss, matthews_corrcoef, brier_score_loss)
tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
print('TN FP FN TP :', tn, fp, fn, tp)
print('accuracy  :', round(accuracy_score(y_test, pred), 3))
print('precision :', round(precision_score(y_test, pred), 3))
print('recall    :', round(recall_score(y_test, pred), 3))
print('F1        :', round(f1_score(y_test, pred), 3))
print('MCC       :', round(matthews_corrcoef(y_test, pred), 3))
print('ROC AUC   :', round(roc_auc_score(y_test, proba), 3))
print('PR AUC    :', round(average_precision_score(y_test, proba), 3))
print('log loss  :', round(log_loss(y_test, proba), 3))
print('Brier     :', round(brier_score_loss(y_test, proba), 3))
```

OUTPUT

```
TN FP FN TP : 52 1 1 89
accuracy  : 0.986
precision : 0.989
recall    : 0.989
F1        : 0.989
MCC       : 0.97
ROC AUC   : 0.998
PR AUC    : 0.999
log loss  : 0.068
Brier     : 0.018
```

Code walkthrough

Lines	What the code does and why
1–3	Import the metric functions; parentheses allow one import over several lines.
4–5	Unpack the 2×2 confusion matrix into its four counts (rows = actual, columns = predicted).
6–10	Label-based metrics receive the hard predictions.
11–14	Probability-based metrics must receive the **probabilities**, not the labels, otherwise the ranking information is lost.

PYTHON Threshold sweep

```python
for t in [0.3, 0.5, 0.7, 0.9]:
    pt = (proba >= t).astype(int)
    print(f't={t}  precision={precision_score(y_test, pt):.3f}'
          f'  recall={recall_score(y_test, pt):.3f}')
```

OUTPUT

```
t=0.3  precision=0.968  recall=1.000
t=0.5  precision=0.989  recall=0.989
t=0.7  precision=0.988  recall=0.933
t=0.9  precision=1.000  recall=0.878
```

Code walkthrough

Lines	What the code does and why
1–4	Evaluate four thresholds. Raising the threshold makes the model pickier: precision goes up while recall goes down.

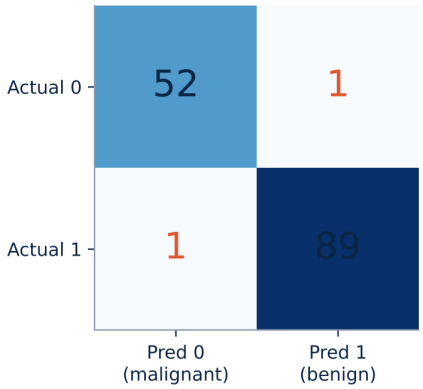


Figure 6. Confusion matrix on the test set; orange cells are the mistakes.

4.6 Checking the assumptions

PYTHON Hosmer–Lemeshow goodness of fit and VIF

```
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor
cols = ['mean radius', 'mean texture', 'mean smoothness', 'mean concavity']
Xs = sm.add_constant(X_train[cols])
logit = sm.Logit(y_train, Xs).fit(disp=0)
def hosmer_lemeshow(y_true, p, groups=10):
    df = pd.DataFrame({'y': np.asarray(y_true), 'p': np.asarray(p)})
    df['bin'] = pd.qcut(df.p, groups, duplicates='drop')
    g = df.groupby('bin', observed=True).agg(obs=('y', 'sum'), exp=('p', 'sum'), n=('y',
        'size'))
    chi2 = (((g.obs - g.exp) ** 2) / (g.exp * (1 - g.exp / g.n))).sum()
    return chi2, 1 - stats.chi2.cdf(chi2, len(g) - 2)
chi2, pval = hosmer_lemeshow(y_train, logit.predict(Xs))
print(f'Hosmer-Lemeshow chi2 = {chi2:.2f}, p = {pval:.3f}  (p > 0.05: fit acceptable)')
Xall = sm.add_constant(X_train)
vif = pd.Series({c: variance_inflation_factor(Xall.values, i) for i, c in
    enumerate(Xall.columns) if c != 'const'})
print(vif.sort_values(ascending=False).head(4).round(0))
```

OUTPUT

```
Hosmer-Lemeshow chi2 = 2.54, p = 0.960  (p > 0.05: fit acceptable)
mean radius        3861.0
mean perimeter     3860.0
worst radius        765.0
worst perimeter     382.0
dtype: float64
```

Code walkthrough

Lines	What the code does and why
1–6	Fit a statsmodels logit on four features. It uses Newton's method and offers full statistical inference; a small subset avoids perfect separation.
7–12	Hosmer–Lemeshow test: sort predictions into 10 risk groups, compare observed and expected positives in each, and sum a chi-square statistic. A large p-value means no evidence of poor fit.
13–14	Apply the test to the training predictions and print the p-value.
15–17	Variance inflation factors on all 30 features. A VIF above 5–10 means a feature is almost fully predictable from the others. Radius and perimeter (and the 'worst' versions) have VIFs in the hundreds or thousands: they are near-duplicates.

Reading the diagnostics

Very large VIFs (thousands for **mean radius** and **mean perimeter**) show that many tumour measurements are near-duplicates. Individual coefficients become unstable, which is the classic reason to use **L2, L1 or ElasticNet regularization** (Section 7).

5. Use Cases

Domain	Question (y = 1)	Typical features
Healthcare	Does the patient have the disease?	Age, BMI, lab values, imaging measurements
Banking	Will the borrower default?	Income, credit score, debt ratio, history
Fraud detection	Is this transaction fraudulent?	Amount, location, time, merchant, device
Marketing	Will the customer respond or buy?	Past clicks, age, offers, channel
Telecom / SaaS	Will the customer churn?	Usage, tenure, complaints, plan
Email / NLP	Is this email spam?	Word counts, links, sender reputation
Human resources	Will the employee leave?	Salary, tenure, satisfaction, overtime
Insurance	Will a claim be filed?	Age, history, vehicle or health profile
Manufacturing	Is the part defective?	Temperature, pressure, machine speed
Education	Will the student pass?	Study hours, attendance, past grades
E-commerce	Will the user click or convert?	Page views, price, device, time of day
Sports analytics	Will the home team win?	Form, injuries, venue, rest days

5.1 What logistic regression does best

- **Risk scoring:** probabilities can be ranked, thresholded and converted into prices or credit limits.
- **Explaining decisions:** odds ratios satisfy regulatory and clinical transparency requirements.
- **Strong baseline and production model** when latency, simplicity and auditability matter.
- **Inference:** p-values and confidence intervals for which factors truly affect the outcome.

5.2 When to use and when to avoid

Use it when	Avoid or upgrade when
Classes are roughly linearly separable	The boundary is strongly non-linear (use polynomial/spline features, trees, boosting)
You need calibrated probabilities	Raw images, audio or text without feature engineering (use deep models)
Interpretability and odds ratios matter	Positives are extremely rare (add class weights, Firth, PR-AUC, threshold tuning)
Data is limited	Rows are clustered or repeated (use mixed or conditional logistic models)
You need a fast, stable baseline	Outcome has 3+ ordered levels (use ordinal logistic)

Common pitfalls

Data leakage: fit scalers and encoders on the training split only (use a Pipeline).
Wrong metric: never judge an imbalanced problem by accuracy alone.
Perfect separation: unregularized maximum likelihood diverges; regularize or use Firth.
Causation: an odds ratio shows association, not that changing the feature will change the outcome.

5.3 Case study: cross-validation and odds ratios

PYTHON Cross-validated AUC and coefficient ranking

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(5, shuffle=True, random_state=42)
auc = cross_val_score(model, X_train, y_train, cv=cv, scoring='roc_auc')
print('AUC per fold:', auc.round(3))
print(f'mean = {auc.mean():.3f}   std = {auc.std():.3f}')
coef = pd.Series(model[-1].coef_[0], index=X.columns)
print(coef.sort_values().head(3).round(2))
print(coef.sort_values().tail(3).round(2))
print('largest odds ratio per +1 std:', round(float(np.exp(coef.abs().max())), 2))
```

OUTPUT

```
AUC per fold: [0.988 0.996 0.998 0.999 0.999]
mean = 0.996   std = 0.004
worst texture    -1.25
radius error     -1.07
worst symmetry   -0.96
dtype: float64
symmetry error       0.43
compactness error    0.62
mean compactness     0.69
dtype: float64
largest odds ratio per +1 std: 3.49
```

Code walkthrough

Lines	What the code does and why
1–3	Stratified 5-fold cross-validation keeps the class balance in every fold. The whole pipeline is re-fitted per fold, so scaling never leaks.
4–5	A high mean and a small standard deviation indicate a strong and stable model.
6–8	Read the weights from the last pipeline step (`model[-1]`). Features are standardized, so sizes are comparable: negative weights push towards class 0 (malignant), positive towards class 1 (benign).
9	exp(coefficient) converts a weight into an odds ratio per one standard-deviation increase.

PYTHON Statistical inference with statsmodels

```python
print(logit.summary2().tables[1].round(3))
print(np.exp(logit.params).round(3))
```

OUTPUT

```
                  Coef.  Std.Err.      z  P>|z|   [0.025  0.975]
const             45.875     6.564  6.989    0.0   33.010  58.741
mean radius       -1.440     0.212 -6.805    0.0   -1.854  -1.025
mean texture      -0.414     0.078 -5.345    0.0   -0.566  -0.262
mean smoothness -159.547    30.397 -5.249    0.0 -219.125 -99.969
mean concavity   -16.898     4.614 -3.662    0.0  -25.941  -7.854
const              8.384280e+19
mean radius        2.370000e-01
mean texture       6.610000e-01
```

```
mean smoothness    0.000000e+00
mean concavity     0.000000e+00
dtype: float64
```

Code walkthrough

Lines	What the code does and why
1	The summary table lists each coefficient, its standard error, z-score, p-value and 95% confidence interval.
2	Exponentiate the coefficients to obtain odds ratios; values below 1 reduce the odds of the benign class.

6. Python Code Structure for Classification Projects

Every scikit-learn model follows one small interface, which is why Logistic Regression can be replaced by SGDClassifier or a random forest by changing a single line. Understanding this structure is the key to clean, leak-free machine-learning code.

6.1 The estimator API for classifiers

Step / member	What it does	Example
Instantiate	Create the model and set hyper-parameters (nothing is learned yet)	`m = LogisticRegression(C=1.0)`
.fit(X, y)	Learn the weights from training data; returns the model	`m.fit(X_train, y_train)`
.predict(X)	Hard class labels using a 0.5 threshold	`m.predict(X_test)`
.predict_proba(X)	Probability for every class; columns follow `classes_`	`m.predict_proba(X_test)[:, 1]`
.decision_function(X)	The raw score z (log-odds) before the sigmoid	`m.decision_function(X_test)`
.score(X, y)	Default metric: accuracy	`m.score(X_test, y_test)`
.coef_ / .intercept_	Learned weights; shape (1, p) for binary, (K, p) for K classes	`m.coef_.shape`
.classes_	Class labels in the order used by predict_proba	`m.classes_`

6.2 Key LogisticRegression parameters

Parameter	Meaning	Typical choice
C	Inverse regularization strength (1/λ); smaller = stronger penalty	Tune on a log grid: 0.001 … 100
l1_ratio	0 = L2, 1 = L1, in between = ElasticNet (scikit-learn 1.8+)	0 by default
solver	Optimization algorithm (see below)	lbfgs for L2; saga for L1/ElasticNet
class_weight	Re-weights classes; 'balanced' for imbalanced data	None or 'balanced'
max_iter	Maximum optimizer iterations; raise it if you see a convergence warning	1000
fit_intercept	Learn β_0	True

Solver	Penalties	Best for
lbfgs	L2, none	Default; small/medium data, multiclass softmax
liblinear	L1, L2 (one-vs-rest)	Small data, L1 selection
newton-cg / newton-cholesky	L2	Accurate, many samples, few features
sag / saga	L2 (sag); L1, L2, ElasticNet (saga)	Large datasets; saga is the most flexible

6.3 Recommended project workflow

#	Stage	Python tool	Notes
1	Load and inspect	pandas: read_csv, info, value_counts	Check class balance, missing values, leakage columns
2	Split first	train_test_split(stratify=y)	Split *before* fitting anything
3	Preprocess	Pipeline + StandardScaler, OneHotEncoder, SimpleImputer	Put every step inside the pipeline
4	Fit and tune	GridSearchCV or LogisticRegressionCV	Choose C and the threshold with cross-validation
5	Evaluate	Confusion matrix, ROC/PR, log loss, calibration	Touch the test set once
6	Interpret and deploy	coef_, odds ratios, joblib.dump	Save the whole pipeline, not just the model

6.4 A reusable evaluation helper

PYTHON One function to compare any classifier

```python
def evaluate(name, m):
    m.fit(X_train, y_train)
    pr = m.predict_proba(X_test)[:, 1]
    lab = (pr >= 0.5).astype(int)
    return {'model': name, 'acc': accuracy_score(y_test, lab), 'f1': f1_score(y_test, lab),
            'auc': roc_auc_score(y_test, pr), 'logloss': log_loss(y_test, pr)}
from sklearn.linear_model import SGDClassifier
rows = [evaluate('Logistic', make_pipeline(StandardScaler(),
    LogisticRegression(max_iter=1000))),
        evaluate('SGD log-loss', make_pipeline(StandardScaler(),
            SGDClassifier(loss='log_loss', random_state=0)))]
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

OUTPUT

```
       model   acc    f1   auc  logloss
    Logistic 0.986 0.989 0.998    0.068
SGD log-loss 0.965 0.972 0.997    0.443
```

Code walkthrough

Lines	What the code does and why
1–6	The helper fits any estimator that supports `predict_proba`, converts probabilities to labels, and returns a dictionary of metrics. Because all scikit-learn classifiers share the same interface, one function serves every model in this document.
7–9	Call the helper for two models wrapped in pipelines. Adding a model means adding one line.
10	Turn the list of dictionaries into a tidy comparison table.

Suggested folder layout for a professional repository:

```
logistic-regression-project/
|-- data/            raw and processed datasets (never edit raw files)
|-- notebooks/       01_eda.ipynb, 02_modeling.ipynb
|-- src/
|   |-- features.py  cleaning, encoding, feature engineering
|   |-- train.py     builds the Pipeline, cross-validates, saves model
|   `-- evaluate.py  metrics, ROC/PR plots, calibration, threshold choice
|-- models/          saved pipelines (joblib)
|-- requirements.txt pinned library versions
`-- README.md        problem, results, how to run
```

7. Regularized Logistic Regression: L2, L1 and ElasticNet

With many features, correlated features or little data, plain maximum likelihood overfits: coefficients become huge and probabilities extreme. In the worst case of **perfect separation** the optimum does not exist at all and the weights diverge. Adding a penalty on the coefficients fixes all of these problems.

$$J_{reg}(\boldsymbol{\beta}) = -\tfrac{1}{n}\sum_{i}[y_i \ln p_i + (1-y_i)\ln(1-p_i)] + \lambda \cdot \text{Penalty}(\boldsymbol{\beta})$$

Model	Penalty	Effect on coefficients	Selects features?	Bayesian view
L2 (Ridge)	$\lambda/2\ \Sigma\beta^2$	Shrinks all weights smoothly	No	Gaussian prior

Model	Penalty	Effect on coefficients	Selects features?	Bayesian view
L1 (Lasso)	$\lambda \Sigma \lvert\beta\rvert$	Shrinks and sets some exactly to 0	Yes	Laplace prior
ElasticNet	Mix of L1 and L2	Sparse yet stable with correlated features	Yes	Mixed prior

- **Standardize features first:** the penalty is scale-sensitive.
- **C = 1/λ:** small C means strong regularization; tune it with cross-validation.
- **L2** handles multicollinearity by sharing weight among correlated features; **L1** picks a sparse subset; **ElasticNet** keeps correlated groups together and works when p > n.

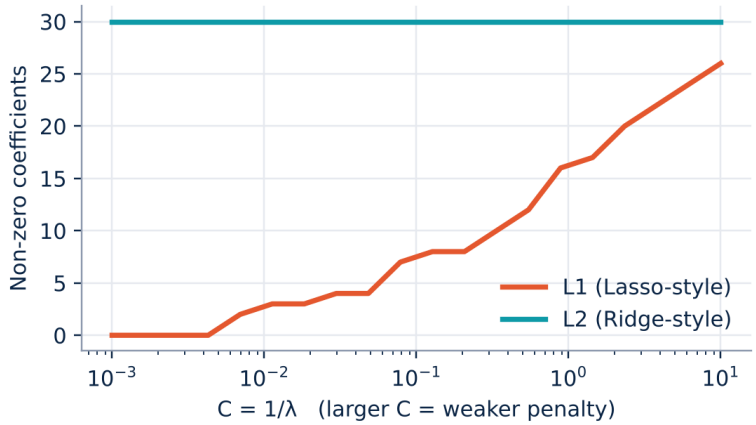


Figure 7. Number of non-zero coefficients as the penalty weakens (C grows): L1 stays sparse, L2 never reaches zero.

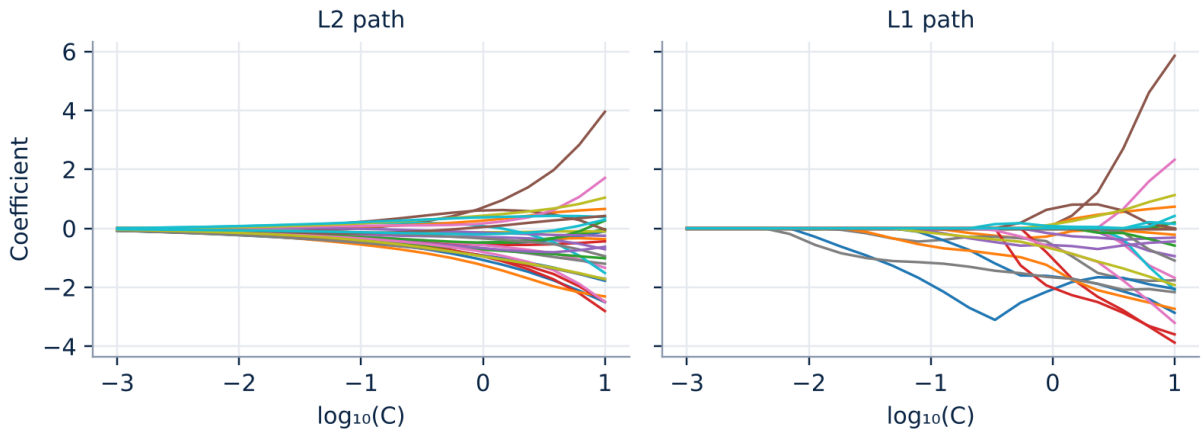


Figure 8. Coefficient paths on the breast-cancer data: L2 shrinks smoothly; L1 drops coefficients to exactly zero.

PYTHON L2 vs L1 vs ElasticNet

```python
from sklearn.metrics import roc_auc_score
models = {
  'L2 (ridge)':  LogisticRegression(l1_ratio=0, C=0.3),
  'L1 (lasso)':  LogisticRegression(l1_ratio=1, solver='saga', C=0.3, max_iter=5000),
  'ElasticNet':  LogisticRegression(l1_ratio=0.5, solver='saga', C=0.3, max_iter=5000)}
for name, m in models.items():
    p = make_pipeline(StandardScaler(), m).fit(X_train, y_train)
    nz = int((p[-1].coef_ != 0).sum())
    a = roc_auc_score(y_test, p.predict_proba(X_test)[:, 1])
    print(f'{name:11s} non-zero coefs = {nz:2d}   test AUC = {a:.3f}')
```

OUTPUT

```
L2 (ridge)  non-zero coefs = 30   test AUC = 0.997
L1 (lasso)  non-zero coefs = 10   test AUC = 0.998
```

```
ElasticNet  non-zero coefs = 21   test AUC = 0.997
```

Code walkthrough

Lines	What the code does and why
1–5	The three models differ only in `l1_ratio` (0 = pure L2, 1 = pure L1). L1 and ElasticNet need the `saga` solver. C = 0.3 is fairly strong regularization. (Older scikit-learn versions use `penalty='l1'/'l2'/'elasticnet'` instead.)
6–10	Fit each model in a pipeline, count the non-zero coefficients and score test AUC. L1 keeps about a third of the features at essentially no loss in accuracy.

8. More Models in the Logistic Family

Logistic regression is a whole family of models. Each variant below addresses a specific weakness: imbalance, more than two classes, ordered outcomes, curved boundaries, different link functions, tiny samples, grouped data, uncertainty, huge data, probability quality and feature selection. All examples reuse the data and variables defined earlier.

Problem	Model to reach for	Section
Rare positive class	Class-weighted, sample-weighted logistic	8.1
3+ unordered classes	Multinomial softmax; One-vs-Rest	8.2, 8.3
Several labels per row	Multilabel (One-vs-Rest)	8.3
Ordered classes	Ordinal / proportional-odds logit	8.4
Curved boundary	Polynomial, spline (GAM-style), kernel logistic	8.5, 8.6
Different link function	Probit, complementary log-log	8.7
Perfect separation, tiny samples	Firth penalized logistic	8.8
Matched or grouped data	Conditional logistic	8.9
Need uncertainty	Bayesian logistic (Laplace)	8.10
Millions of rows / streaming	SGDClassifier, partial_fit	8.11
Automatic tuning	LogisticRegressionCV, GridSearchCV	8.12
Better probabilities	Calibration (Platt, isotonic)	8.13
Select features	SelectFromModel, RFECV	8.14
Related linear classifiers	LDA, Naive Bayes, SVM, Ridge classifier, Perceptron	8.15

8.1 Class-weighted logistic regression (imbalanced data)

When positives are rare, the model can reach a low loss by mostly ignoring them. **class_weight='balanced'** multiplies each sample's loss by *n / (K × class_count)*, so mistakes on the rare class cost more. Other tools: per-row `sample_weight`, resampling (SMOTE, undersampling), threshold tuning and PR-AUC for evaluation.

PYTHON Effect of class_weight on a rare class

```python
rng = np.random.RandomState(0)
idx0 = np.where(y_train == 0)[0]
idx1 = np.where(y_train == 1)[0]
keep = np.r_[rng.choice(idx0, 20, replace=False), idx1]
Xi, yi = X_train.iloc[keep], y_train.iloc[keep]
print('imbalanced counts:', yi.value_counts().to_dict())
for cw in [None, 'balanced']:
    m = make_pipeline(StandardScaler(), LogisticRegression(class_weight=cw, max_iter=1000))
    pr_ = m.fit(Xi, yi).predict(X_test)
    print(f'class_weight={str(cw):9s} recall(malignant) = {recall_score(y_test, pr_,
        pos_label=0):.3f}')
```

OUTPUT

```
imbalanced counts: {1: 267, 0: 20}
class_weight=None      recall(malignant) = 0.868
class_weight=balanced  recall(malignant) = 0.925
```

Code walkthrough

Lines	What the code does and why
1–6	Keep only 20 malignant (class 0) cases in the training set to simulate a rare class.
7–10	Compare recall on the rare class (`pos_label=0`). Balanced weights recover more of the rare malignant cases at a small cost in precision.

8.2 Multinomial logistic regression (softmax)

For K > 2 unordered classes the softmax model learns one weight vector per class and outputs probabilities that sum to 1. It is the default in scikit-learn for multiclass problems and equals the output layer of a neural-network classifier.

PYTHON Softmax from scratch and with scikit-learn (iris, 3 classes)

```python
def softmax(Z):
    E = np.exp(Z - Z.max(axis=1, keepdims=True))
    return E / E.sum(axis=1, keepdims=True)
from sklearn.datasets import load_iris
Xi3, yi3 = load_iris(return_X_y=True)
Zi = np.c_[np.ones(len(Xi3)), StandardScaler().fit_transform(Xi3)]
Y1 = np.eye(3)[yi3]
W = np.zeros((5, 3))
for it in range(1000):
    W -= 0.5 * Zi.T @ (softmax(Zi @ W) - Y1) / len(Zi)
print('from-scratch accuracy:', (softmax(Zi @ W).argmax(axis=1) == yi3).mean().round(3))
Xa, Xt, ya, yt = train_test_split(Xi3, yi3, test_size=0.3, random_state=0, stratify=yi3)
soft = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500)).fit(Xa, ya)
print('sklearn accuracy :', round(soft.score(Xt, yt), 3))
print('coef_ shape       :', soft[-1].coef_.shape)
print('probabilities row0:', soft.predict_proba(Xt[:1]).round(3))
```

OUTPUT

```
from-scratch accuracy: 0.973
sklearn accuracy : 0.978
coef_ shape       : (3, 4)
probabilities row0: [[0.    0.036 0.964]]
```

Code walkthrough

Lines	What the code does and why
1–3	Softmax turns K scores into K probabilities. Subtracting the row maximum keeps the exponentials numerically stable.
4–10	One-hot encode the labels and run gradient descent on the weight matrix W (5 inputs × 3 classes). The gradient has the same elegant form $X^T(P - Y)/n$ as binary logistic regression.
11	The predicted class is the one with the largest probability.
12–16	With three classes scikit-learn fits a softmax model: `coef_` has one row of weights per class, and the three probabilities of a row sum to 1.

8.3 One-vs-Rest and multilabel classification

- **One-vs-Rest (OvR):** train K independent binary models, each class against all the others, and pick the highest score. Simple and parallel, but probabilities are not coupled.
- **Multilabel:** each row can have several labels at once (tags, symptoms). OvR with one logistic model per label is the standard approach.

PYTHON OvR for multiclass and multilabel problems

```python
from sklearn.multiclass import OneVsRestClassifier
ovr = make_pipeline(StandardScaler(), OneVsRestClassifier(LogisticRegression())).fit(Xa, ya)
print('OvR accuracy:', round(ovr.score(Xt, yt), 3))
from sklearn.datasets import make_multilabel_classification
from sklearn.metrics import f1_score
Xl, Yl = make_multilabel_classification(300, n_classes=4, random_state=0)
ml = OneVsRestClassifier(LogisticRegression()).fit(Xl, Yl)
print('label matrix shape:', ml.predict(Xl).shape)
print('micro F1 (train):', round(f1_score(Yl, ml.predict(Xl), average='micro'), 3))
```

OUTPUT

```
OvR accuracy: 0.889
label matrix shape: (300, 4)
micro F1 (train): 0.83
```

Code walkthrough

Lines	What the code does and why
1–3	Wrap the binary model in `OneVsRestClassifier` to train one model per class. Here OvR is clearly lower than the joint softmax model, because three independent binary models cope less well with the overlapping versicolor and virginica classes.
4–7	Create data where each row has up to four labels, then fit one logistic model per label.
8–9	The prediction is a 0/1 matrix with one column per label; micro-averaged F1 pools all label decisions.

8.4 Ordinal logistic regression (proportional odds)

When classes have a natural order (low < medium < high), ignoring the order wastes information. The **cumulative-link (proportional-odds) model** uses one set of slopes and one threshold per class boundary:

$$P(y \le k \mid \mathbf{x}) = \sigma\big(\theta_k - \boldsymbol{\beta}^T \mathbf{x}\big), \qquad \theta_1 < \theta_2 < \cdots < \theta_{K-1}$$

PYTHON Ordinal model with statsmodels (disease severity: low / mid / high)

```python
from statsmodels.miscmodels.ordinal_model import OrderedModel
from sklearn.datasets import load_diabetes
d = load_diabetes(as_frame=True)
level = pd.qcut(d.target, 3, labels=['low', 'mid', 'high'])
om = OrderedModel(level, d.data[['bmi', 'bp', 's5']], distr='logit')
res = om.fit(method='bfgs', disp=0)
print(res.params.round(2))
print(res.predict(d.data[['bmi', 'bp', 's5']].iloc[:3]).round(2))
```

OUTPUT

```
bmi         16.03
bp           9.55
s5          19.60
low/mid     -1.10
mid/high     0.76
dtype: float64
      0     1     2
0  0.06  0.30  0.63
1  0.79  0.18  0.03
2  0.14  0.44  0.42
```

Code walkthrough

Lines	What the code does and why
1–4	Convert the diabetes progression score into three ordered groups with quantile binning.
5–7	Fit the ordered logit. The slopes (bmi, bp, s5) are positive when they raise severity; 'low/mid' and 'mid/high' are the learned cut-points (thresholds).
8	predict returns one probability per ordered class for each row; each row sums to 1.

8.5 Polynomial and spline logistic regression

The boundary is linear in the features, but you can **add squares, interactions or spline basis functions** so that the boundary bends. The model is still ordinary logistic regression on transformed inputs, with the same convex loss. Splines (a GAM-style approach) give smooth, local curvature without the wild behaviour of high-degree polynomials.

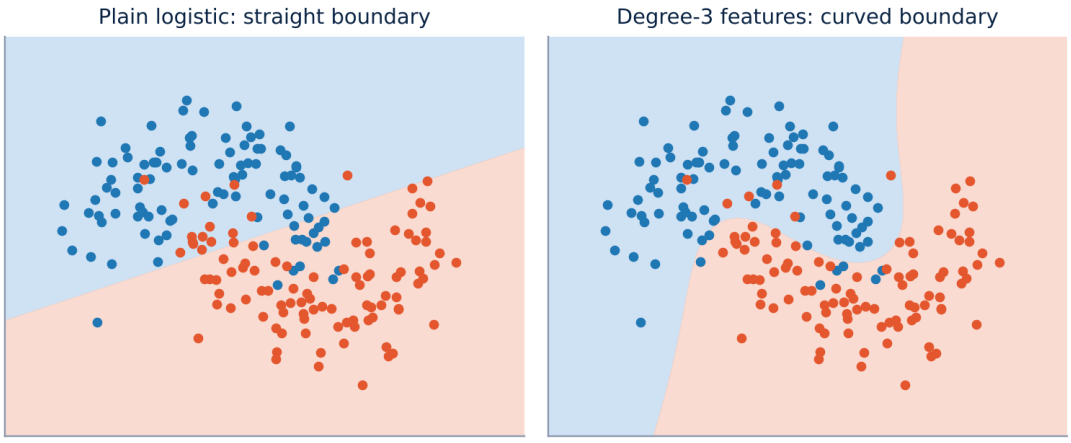


Figure 9. Adding polynomial features lets logistic regression separate the two moons.

PYTHON Plain vs polynomial vs spline features on the two-moons data

```python
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures, SplineTransformer
from sklearn.model_selection import cross_val_score
Xm, ym = make_moons(300, noise=0.25, random_state=0)
variants = {
  'plain':      make_pipeline(StandardScaler(), LogisticRegression()),
  'polynomial': make_pipeline(PolynomialFeatures(3), StandardScaler(),
      LogisticRegression(C=10, max_iter=2000)),
  'spline':     make_pipeline(SplineTransformer(n_knots=5, degree=3),
      LogisticRegression(C=10))}
for name, m in variants.items():
    print(f'{name:10s} CV accuracy = {cross_val_score(m, Xm, ym, cv=5).mean():.3f}')
```

OUTPUT

```
plain      CV accuracy = 0.833
polynomial CV accuracy = 0.947
spline     CV accuracy = 0.927
```

Code walkthrough

Lines	What the code does and why
1–4	Generate two interleaved half-moons that no straight line can separate.
5–8	Same classifier, different feature maps: degree-3 polynomial terms, or cubic B-spline basis functions with 5 knots per feature.
9–10	Cross-validated accuracy rises sharply once the boundary can curve.

8.6 Kernel logistic regression (Nystroem approximation)

Exact kernel logistic regression is expensive ($O(n^3)$). The **Nystroem** method approximates an RBF kernel feature map with a few hundred components, after which ordinary logistic regression gives a non-linear classifier that scales to large data.

PYTHON RBF features + logistic regression

```python
from sklearn.kernel_approximation import Nystroem
kern = make_pipeline(StandardScaler(), Nystroem(gamma=0.5, n_components=60,
    random_state=0), LogisticRegression())
print('kernel logistic CV accuracy =', round(cross_val_score(kern, Xm, ym, cv=5).mean(), 3))
```

OUTPUT

```
kernel logistic CV accuracy = 0.91
```

Code walkthrough

Lines	What the code does and why
1–2	Nystroem maps each point to 60 kernel similarities (RBF with width gamma); the logistic layer is linear in these new features.
3	A smooth non-linear boundary without hand-crafting polynomial terms.

8.7 Probit and complementary log-log models

The **link function** converts the linear score into a probability. Logit (the sigmoid) is standard; **probit** uses the normal CDF and has slightly lighter tails; **complementary log-log** is asymmetric and suits events whose probability approaches 1 slowly (for example the chance of at least one event in a time interval). Probit coefficients are typically 1.6–1.9 times smaller than logit coefficients.

$$\text{Logit: } p = \frac{1}{1+e^{-z}} \qquad \text{Probit: } p = \Phi(z) \qquad \text{cloglog: } p = 1 - e^{-e^{z}}$$

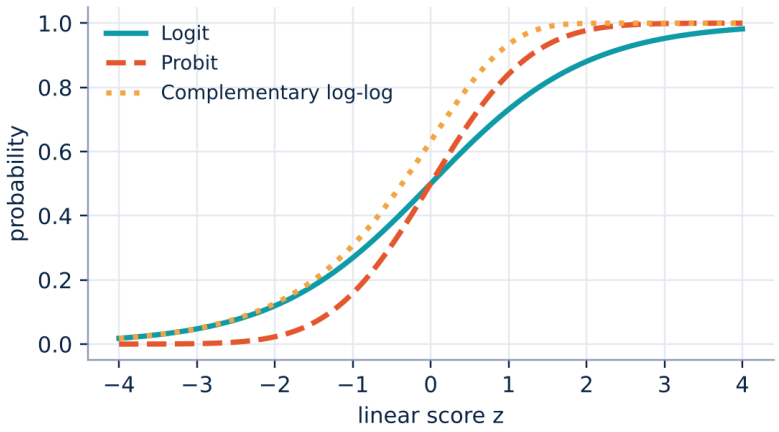


Figure 10. Three link functions: logit and probit are symmetric; cloglog is not.

PYTHON Binomial GLMs with different links

```python
links = {'logit': sm.families.links.Logit(), 'probit': sm.families.links.Probit(),
         'cloglog': sm.families.links.CLogLog()}
fits = {k: sm.GLM(y_train, Xs, family=sm.families.Binomial(link=l)).fit() for k, l in
    links.items()}
print((fits['logit'].params / fits['probit'].params).round(2))
Xte_s = sm.add_constant(X_test[cols])
diff = np.abs(fits['logit'].predict(Xte_s) - fits['probit'].predict(Xte_s)).max()
print('max |logit - probit| probability:', round(float(diff), 3))
```

OUTPUT

```
const              1.91
mean radius        1.89
mean texture       1.91
mean smoothness    1.91
mean concavity     1.92
dtype: float64
max |logit - probit| probability: 0.042
```

Code walkthrough

Lines	What the code does and why
1–3	A logistic regression is a Binomial generalized linear model. Changing only the `link` turns it into a probit or cloglog model.
4	Dividing logit by probit coefficients gives ratios of about 1.9 for every term, in line with the theoretical factor of roughly 1.6–1.8 ($\pi/\sqrt{3} \approx 1.81$).
5–7	The two models give almost identical probabilities, so the choice rarely matters for prediction.

8.8 Firth penalized logistic regression

With **perfect or quasi-perfect separation**, small samples or very rare events, ordinary maximum likelihood is biased or even undefined. **Firth's method** adds a Jeffreys-prior penalty (half the log-determinant of the information matrix) to the likelihood, producing finite, bias-reduced estimates without any tuning parameter.

$$\ell^{*}(\boldsymbol{\beta}) = \ell(\boldsymbol{\beta}) + \tfrac{1}{2}\ln\left|\mathbf{X}^{T}\mathbf{W}\mathbf{X}\right|$$

PYTHON Firth's method vs plain maximum likelihood on separable data

```python
xs = np.array([1, 2, 3, 4, 5, 6.]); ys = np.array([0, 0, 0, 1, 1, 1]); Xf =
    np.c_[np.ones(6), xs]
plain = LogisticRegression(C=1e8, max_iter=10000).fit(xs[:, None], ys)
print('plain ML  : intercept', plain.intercept_.round(1), ' slope', plain.coef_.round(1))
def firth(X, y, iters=50):
    b = np.zeros(X.shape[1])
    for _ in range(iters):
        p = 1 / (1 + np.exp(-X @ b)); W = p * (1 - p)
        inv = np.linalg.inv(X.T @ (X * W[:, None]))
        h = np.einsum('ij,jk,ik->i', X, inv, X) * W
        step = inv @ (X.T @ (y - p + h * (0.5 - p)))
        b += step
        if np.abs(step).max() < 1e-8: break
    return b
print('Firth     :', firth(Xf, ys).round(3))
```

OUTPUT

```
plain ML  : intercept [-52.9]  slope [[15.2]]
Firth     : [-3.951  1.129]
```

Code walkthrough

Lines	What the code does and why
1–3	A perfectly separable toy set: every x ≥ 4 is class 1. Without a penalty the weights blow up (limited only by the iteration limit).
4–13	Firth's algorithm is Newton's method with a modified score: the residual y − p is adjusted by h(0.5 − p), where h are the leverages (diagonal of the hat matrix). This shrinks estimates toward 0.5 and keeps them finite.
14	Firth returns moderate, usable coefficients (about −3.95 and 1.13) where plain ML returns extreme ones.

8.9 Conditional logistic regression

In **matched case–control studies** and other designs with many small groups, each group has its own baseline risk. **Conditional logistic regression** conditions on the group so that these nuisance intercepts cancel, estimating only the slope parameters that vary within groups.

PYTHON Conditional logit with group-specific baselines

```python
from statsmodels.discrete.conditional_models import ConditionalLogit
rng = np.random.RandomState(0)
g = np.repeat(np.arange(60), 4); xg = rng.normal(size=240)
base = rng.normal(size=60)[g]
yg = (rng.uniform(size=240) < 1 / (1 + np.exp(-(base + 1.0 * xg)))).astype(int)
cl = ConditionalLogit(yg, xg[:, None], groups=g).fit(disp=0)
print('conditional logit estimate:', cl.params.round(2), '(true value 1.0)')
```

OUTPUT

```
conditional logit estimate: [1.02] (true value 1.0)
```

Code walkthrough

Lines	What the code does and why
1–5	Simulate 60 groups of 4 rows. Each group has its own random baseline, but the true effect of the feature is exactly 1.0.
6–7	The estimator recovers the slope without ever estimating the 60 baselines. (Groups with all-equal outcomes carry no information and are dropped automatically.)

8.10 Bayesian logistic regression (Laplace approximation)

Bayesian logistic regression places a prior on the weights and returns a **posterior distribution** instead of a single estimate, so every prediction comes with uncertainty. The **Laplace approximation** finds the most probable weights (MAP, equal to L2-regularized logistic regression) and approximates the posterior with a Gaussian whose covariance is the inverse Hessian.

$$p(\boldsymbol{\beta} \mid \mathcal{D}) \approx \mathcal{N}\big(\boldsymbol{\beta}_{MAP},\ \mathbf{H}^{-1}\big), \qquad \mathbf{H} = \mathbf{X}^T\mathbf{W}\mathbf{X} + \tau^{-2}\mathbf{I}$$

PYTHON Predictive uncertainty for one patient

```python
from scipy.optimize import minimize
sc4 = StandardScaler().fit(X_train[cols])
Zb = np.c_[np.ones(len(X_train)), sc4.transform(X_train[cols])]
tau2 = 10.0
def neg_log_post(b):
    z = Zb @ b
    return np.sum(np.logaddexp(0, z) - y_train.values * z) + 0.5 * b @ b / tau2
b_map = minimize(neg_log_post, np.zeros(Zb.shape[1]), method='BFGS').x
p = 1 / (1 + np.exp(-Zb @ b_map))
H = Zb.T @ (Zb * (p * (1 - p))[:, None]) + np.eye(len(b_map)) / tau2
cov = np.linalg.inv(H)
draws = np.random.RandomState(1).multivariate_normal(b_map, cov, 2000)
z0 = np.r_[1, sc4.transform(X_test[cols].iloc[[0]])[0]]
probs = 1 / (1 + np.exp(-draws @ z0))
print('P(benign) for test row 0: mean', probs.mean().round(3), ' 95% interval',
    np.percentile(probs, [2.5, 97.5]).round(3))
```

OUTPUT

```
P(benign) for test row 0: mean 0.866  95% interval [0.723 0.947]
```

Code walkthrough

Lines	What the code does and why
1–7	Standardize four features and define the negative log posterior: log loss written stably with `logaddexp`, plus a Gaussian prior (variance tau²) that acts as an L2 penalty.
8–11	Find the MAP weights by optimization, then build the Hessian at that point; its inverse is the approximate posterior covariance.
12–15	Sample 2000 weight vectors from the posterior and push the first test patient through each. The spread of the resulting probabilities is the model's uncertainty about this prediction.

8.11 SGDClassifier and online learning

SGDClassifier(loss='log_loss') trains logistic regression with stochastic gradient descent, one sample or mini-batch at a time. It scales to millions of rows, supports any penalty and learns from streaming data through `partial_fit`. Changing `loss='hinge'` gives a linear SVM. Features must be scaled.

PYTHON SGD logistic regression, including streaming updates

```python
from sklearn.linear_model import SGDClassifier
sgd = make_pipeline(StandardScaler(), SGDClassifier(loss='log_loss', alpha=1e-3,
      max_iter=2000, tol=1e-4, random_state=0)).fit(X_train, y_train)
print('epochs used:', sgd[-1].n_iter_, '  test accuracy:', round(sgd.score(X_test, y_test),
    3))
online = SGDClassifier(loss='log_loss', alpha=1e-3, random_state=0)
sc5 = StandardScaler().fit(X_train)
for chunk in np.array_split(np.arange(len(X_train)), 5):
    online.partial_fit(sc5.transform(X_train.iloc[chunk]), y_train.iloc[chunk], classes=[0,
        1])
print('streamed (5 chunks) accuracy:', round(online.score(sc5.transform(X_test), y_test),
    3))
```

OUTPUT

```
epochs used: 39   test accuracy: 0.986
streamed (5 chunks) accuracy: 0.958
```

Code walkthrough

Lines	What the code does and why
1–4	`loss='log_loss'` makes SGD a logistic regression; `alpha` is the regularization strength (like 1/C). Training stops once improvement falls below `tol`.
5–9	`partial_fit` updates the model with one chunk at a time without keeping all data in memory. The first call must list all possible `classes`.

8.12 Automatic tuning: LogisticRegressionCV and GridSearchCV

PYTHON Let cross-validation choose the regularization

```python
from sklearn.linear_model import LogisticRegressionCV
cv_model = make_pipeline(StandardScaler(),
    LogisticRegressionCV(Cs=10, cv=5, scoring='roc_auc', max_iter=3000)).fit(X_train,
        y_train)
print('chosen C :', cv_model[-1].C_.round(3))
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000)),
    {'logisticregression__C': [0.01, 0.1, 1, 10],
     'logisticregression__class_weight': [None, 'balanced']}, cv=5,
         scoring='f1').fit(X_train, y_train)
print(grid.best_params_, round(grid.best_score_, 3))
```

OUTPUT

```
chosen C : [0.359]
{'logisticregression__C': 0.1, 'logisticregression__class_weight': None} 0.982
```

Code walkthrough

Lines	What the code does and why
1–4	Try 10 values of C on a log grid, score each with 5-fold AUC and refit with the best one; C_ stores the winner.
5–9	GridSearchCV tries every combination of parameters (here 4 × 2). Pipeline parameters are addressed as `stepname__parameter`.

8.13 Probability calibration

A model is **calibrated** when predicted probabilities match observed frequencies. Logistic regression is usually well calibrated; Naive Bayes and SVMs are not. **CalibratedClassifierCV** learns a mapping from raw scores to probabilities, either *sigmoid* (Platt scaling) or *isotonic* regression (flexible, needs more data).

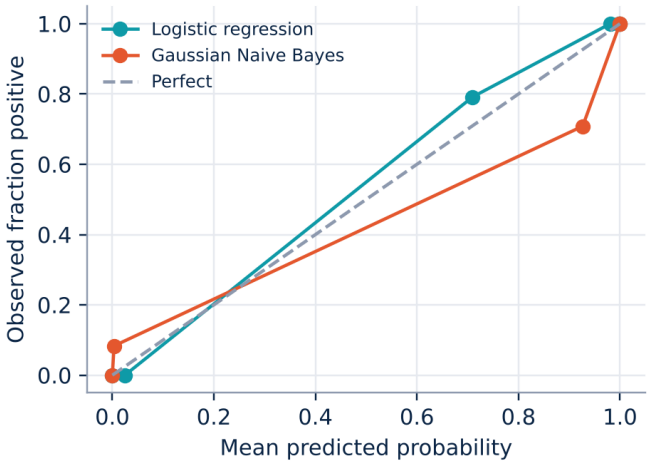


Figure 11. Reliability curve: logistic regression hugs the diagonal; Naive Bayes is over-confident.

PYTHON Calibrating Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
from sklearn.calibration import CalibratedClassifierCV
nb = make_pipeline(StandardScaler(), GaussianNB()).fit(X_train, y_train)
nb_cal = CalibratedClassifierCV(make_pipeline(StandardScaler(), GaussianNB()),
    method='isotonic', cv=5).fit(X_train, y_train)
for name, m in [('logistic', model), ('naive Bayes', nb), ('NB calibrated', nb_cal)]:
    print(f'{name:14s} Brier = {brier_score_loss(y_test, m.predict_proba(X_test)[:,
        1]):.3f}')
```

OUTPUT

```
logistic       Brier = 0.018
naive Bayes    Brier = 0.057
NB calibrated  Brier = 0.042
```

Code walkthrough

Lines	What the code does and why
1–4	Fit Naive Bayes plainly and wrapped in an isotonic calibrator that is trained by 5-fold cross-validation.
5–6	The Brier score (lower is better) shows that calibration improves Naive Bayes (0.057 to 0.042), yet it still does not reach the natively well-calibrated logistic model (0.018).

8.14 Feature selection: SelectFromModel and RFECV

PYTHON Two ways to pick features with logistic regression

```
from sklearn.feature_selection import SelectFromModel, RFECV
sel = make_pipeline(StandardScaler(), SelectFromModel(
    LogisticRegression(l1_ratio=1, solver='saga', C=0.1, max_iter=5000)))
sel.fit(X_train, y_train)
print('L1 keeps', int(sel[-1].get_support().sum()), 'features:',
    list(X.columns[sel[-1].get_support()])[:5], '...')
rfe = RFECV(LogisticRegression(max_iter=3000), step=2, cv=5, scoring='roc_auc')
rfe.fit(StandardScaler().fit_transform(X_train), y_train)
print('RFECV optimal number of features:', rfe.n_features_)
```

OUTPUT

```
L1 keeps 7 features: ['mean concave points', 'radius error', 'worst radius', 'worst texture', 'worst
RFECV optimal number of features: 20
```

Code walkthrough

Lines	What the code does and why
1–5	SelectFromModel keeps the features whose L1-penalized coefficients are non-zero.
6–8	Recursive Feature Elimination repeatedly removes the weakest 2 features and uses cross-validated AUC to choose how many to keep.

8.15 Related linear classifiers

Model	Idea	Compared with logistic regression
Linear Discriminant Analysis	Assumes Gaussian classes with a shared covariance; boundary from Bayes' rule	Same linear boundary; efficient with small data if Gaussian, less robust to outliers
Gaussian Naive Bayes	Assumes features are independent given the class	Very fast, but over-confident probabilities; needs calibration
Linear SVM	Maximizes the margin using hinge loss	No native probabilities (calibrate); robust boundary
Ridge classifier	Least-squares regression on ±1 labels with L2 penalty	Closed-form and fast; no probabilities
Perceptron	Online updates only on mistakes	Oldest linear classifier; no probabilities, no regularization by default
Single-layer neural net	Sigmoid/softmax output on learned features	Logistic regression is its simplest special case

Other specialised models	When they help
Mixed-effects (multilevel) logistic	Repeated measures, students within schools, patients within hospitals
Exact logistic regression	Very small samples where asymptotic p-values are unreliable
Rare-event logistic (King–Zeng)	Positives under ~1%; corrects intercept and probability bias
Stereotype / adjacent-category logit	Ordered outcomes where proportional odds does not hold
Nested and mixed multinomial logit	Choice modelling with related alternatives (transport, marketing)
Group Lasso / fused Lasso logistic	Structured sparsity: one-hot groups, ordered or spatial features
Discrete-time hazard (logistic)	Survival analysis written as repeated binary outcomes per period
Generalized additive logistic (GAM)	Smooth non-linear effects while staying interpretable

9. Model Comparison and How to Choose

All models below were trained on the same 426 training rows of the breast-cancer dataset (30 features) and evaluated on the same 143 held-out rows, with standardized inputs. CV AUC is the 5-fold stratified cross-validated score on the training data (accuracy for models without scores).

PYTHON Compare several variants with the reusable helper

```python
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
S = StandardScaler
rows = [
  evaluate('L2', make_pipeline(S(), LogisticRegression(max_iter=2000))),
  evaluate('L1', make_pipeline(S(), LogisticRegression(l1_ratio=1, solver='saga', C=0.3,
      max_iter=5000))),
  evaluate('Balanced', make_pipeline(S(), LogisticRegression(class_weight='balanced',
      max_iter=2000))),
  evaluate('LDA', make_pipeline(S(), LinearDiscriminantAnalysis())),
  evaluate('Naive Bayes', make_pipeline(S(), GaussianNB()))]
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

OUTPUT

```
      model   acc    f1   auc  logloss
         L2 0.986 0.989 0.998    0.068
         L1 0.965 0.973 0.998    0.091
   Balanced 0.965 0.972 0.998    0.076
        LDA 0.951 0.962 0.994    0.112
Naive Bayes 0.937 0.951 0.988    0.409
```

Code walkthrough

Lines	What the code does and why
1–8	Reuse the `evaluate` helper from Section 6 for five models. The same train/test split guarantees a fair comparison.
9	Print one tidy table of accuracy, F1, AUC and log loss.

#	Model	CV score	Accuracy	F1	Test AUC	Log loss
1	Logistic L2	0.996	0.986	0.989	0.998	0.068
2	Logistic L1	0.994	0.965	0.973	0.998	0.091
3	Logistic balanced weights	0.996	0.965	0.972	0.998	0.076
4	Logistic CV-tuned C	0.996	0.986	0.989	0.998	0.079
5	Logistic ElasticNet	0.995	0.986	0.989	0.997	0.087
6	SGD (log loss)	0.995	0.986	0.989	0.997	0.065
7	Linear SVM (calibrated)	0.995	0.965	0.973	0.995	0.141
8	Nystroem kernel + Logistic	0.993	0.944	0.957	0.995	0.174
9	Ridge classifier	0.955	0.958	0.968	0.995	n/a
10	Random forest (reference)	0.987	0.958	0.967	0.995	0.102
11	LDA	0.992	0.951	0.962	0.994	0.112
12	Polynomial deg 2 + L2	0.995	0.951	0.962	0.994	0.128
13	Perceptron	0.962	0.958	0.966	0.994	n/a

#	Model	CV score	Accuracy	F1	Test AUC	Log loss
14	Gaussian Naive Bayes	0.986	0.937	0.951	0.988	0.409
15	Logistic (almost unregularized)	0.984	0.958	0.967	0.969	0.996

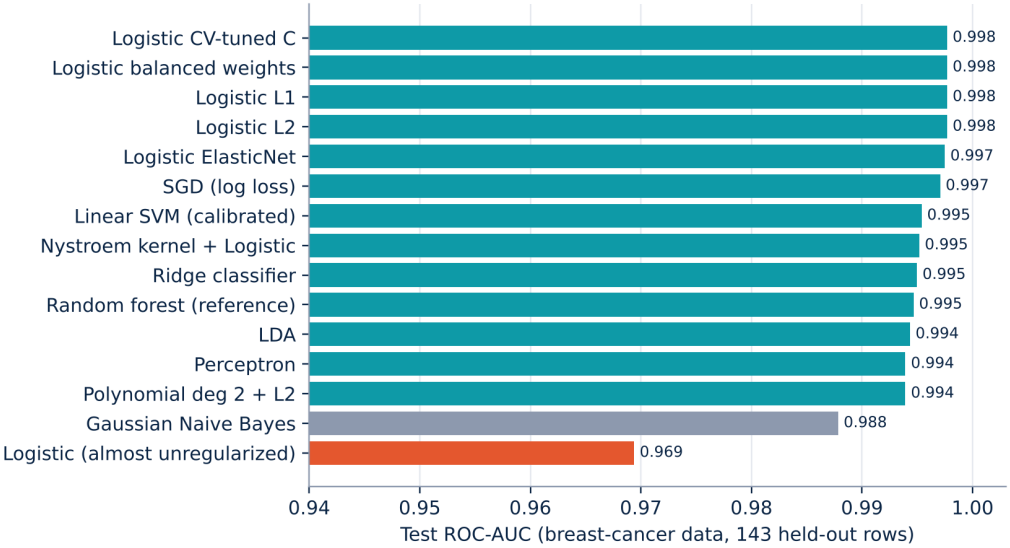


Figure 12. Test ROC-AUC of every model (teal = best group, orange = weakest).

What the numbers tell us

The regularized logistic variants (L2, L1, ElasticNet, CV-tuned, balanced, SGD) lead with test AUC ≈ 0.997–0.998 and accuracy up to 0.986.

The almost **unregularized** model is clearly worse (AUC 0.969) and its log loss is about 1.0 versus about 0.07: its probabilities are over-confident, even though its accuracy looks acceptable. **Regularization matters more than the exact penalty.**

Naive Bayes ranks lowest on log loss (poor calibration), and the non-linear models (polynomial, kernel, random forest) bring no gain on this almost linearly separable dataset.

With only 143 test rows, differences of 0.01 in accuracy equal 1–2 patients; rely on cross-validation, not one split.

9.1 Decision guide

Situation	Recommended model
Few clean features, need p-values and odds ratios	Plain logit (statsmodels)
Many or correlated features, want stable predictions	L2 logistic regression
Want automatic feature selection	L1 logistic regression, SelectFromModel, RFECV
Many correlated features and want selection	ElasticNet logistic regression
Rare positive class	class_weight='balanced', threshold tuning, PR-AUC, Firth for tiny samples
3+ unordered classes	Multinomial softmax
Ordered classes	Ordinal (proportional-odds) logit
Several labels per observation	One-vs-Rest multilabel
Curved decision boundary	Polynomial or spline features, Nystroem kernel features
Need uncertainty on predictions	Bayesian logistic regression
Grouped or matched observations	Conditional or mixed-effects logistic
Millions of rows or streaming data	SGDClassifier with partial_fit
Probabilities must be trustworthy	Logistic regression, or calibrate other models

9.2 Master comparison of the penalized family

Property	Plain	L2	L1	ElasticNet
Penalty	None	$\lambda/2\ \Sigma\beta^2$	$\lambda\ \Sigma\lvert\beta\rvert$	Mix of both
Feature selection	No	No	Yes	Yes
Handles perfect separation	No (diverges)	Yes	Yes	Yes
Multicollinearity	Poor	Excellent	Moderate	Excellent
scikit-learn setting	C=1e6	l1_ratio=0	l1_ratio=1, saga	0<l1_ratio<1, saga
Hyper-parameters	0	1 (C)	1 (C)	2 (C, l1_ratio)

10. Cheat Sheet and Summary

Topic	Remember
Model	$p = \sigma(\beta_0 + \beta^T x)$; the log-odds are linear; $\exp(\beta)$ is an odds ratio
Loss	Log loss (binary cross-entropy), convex; gradient = $X^T(p - y)/n$; solved iteratively (L-BFGS, Newton, SGD)
Decision	Predict 1 when $p \geq$ threshold; choose the threshold from error costs
Metrics	Confusion matrix, precision, recall, F1, MCC, ROC-AUC, PR-AUC, log loss, Brier, calibration
Assumptions	Independent rows, linear logit, no severe multicollinearity, enough events per feature
Overfitting / separation	Regularize ($C = 1/\lambda$), standardize features, tune C by cross-validation; Firth for tiny samples
Imbalance	class_weight, sample_weight, resampling, PR-AUC, threshold tuning
Beyond binary	Softmax (multiclass), ordinal logit, One-vs-Rest (multilabel)
Non-linear	Polynomial, spline or kernel (Nystroem) features
Code habits	Stratified split first, scale inside a Pipeline, evaluate once on the test set, save the pipeline

Learning roadmap

- Re-implement sigmoid, log loss and gradient descent from scratch (Sections 2–3).
- Evaluate with the full metric set and choose a threshold (Section 4).
- Study odds ratios and diagnostics with statsmodels (Sections 4–5).
- Compare L2, L1 and ElasticNet with cross-validation (Section 7).
- Try multinomial, ordinal, Firth, Bayesian and calibrated variants (Section 8).
- Package the final model as a Pipeline and save it with `joblib` (Section 6).

About the author

These notes were prepared by **Chandan Chaudhari**.

GitHub: github.com/chandanc5525

Feedback, corrections and stars on the repository are always welcome.